

웹 프로그래밍

14. 웹 서버 구축

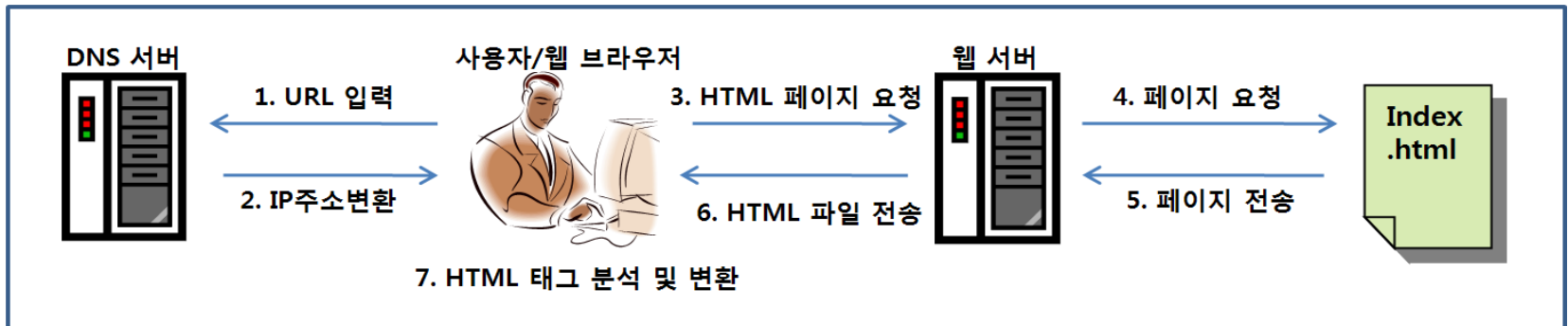
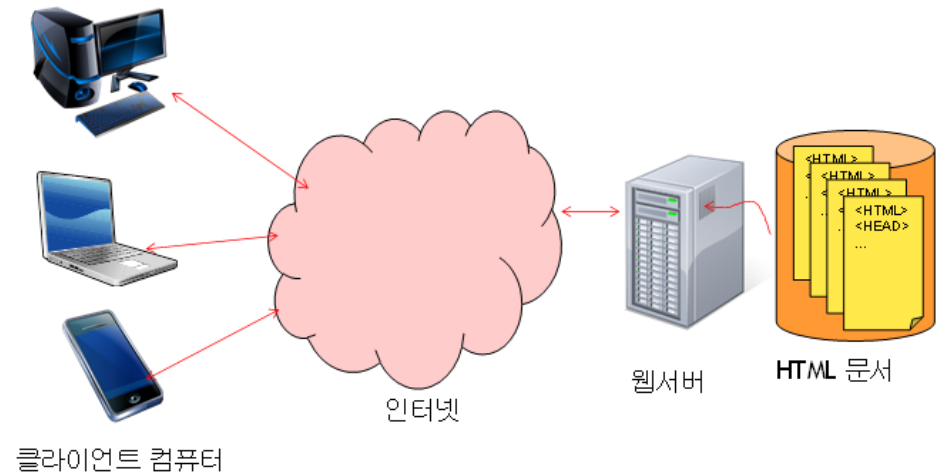
웹 동작 방식

● 웹 서버

- HTML 문서들 저장
- 클라이언트들의 요청에 따른 웹 문서 제공

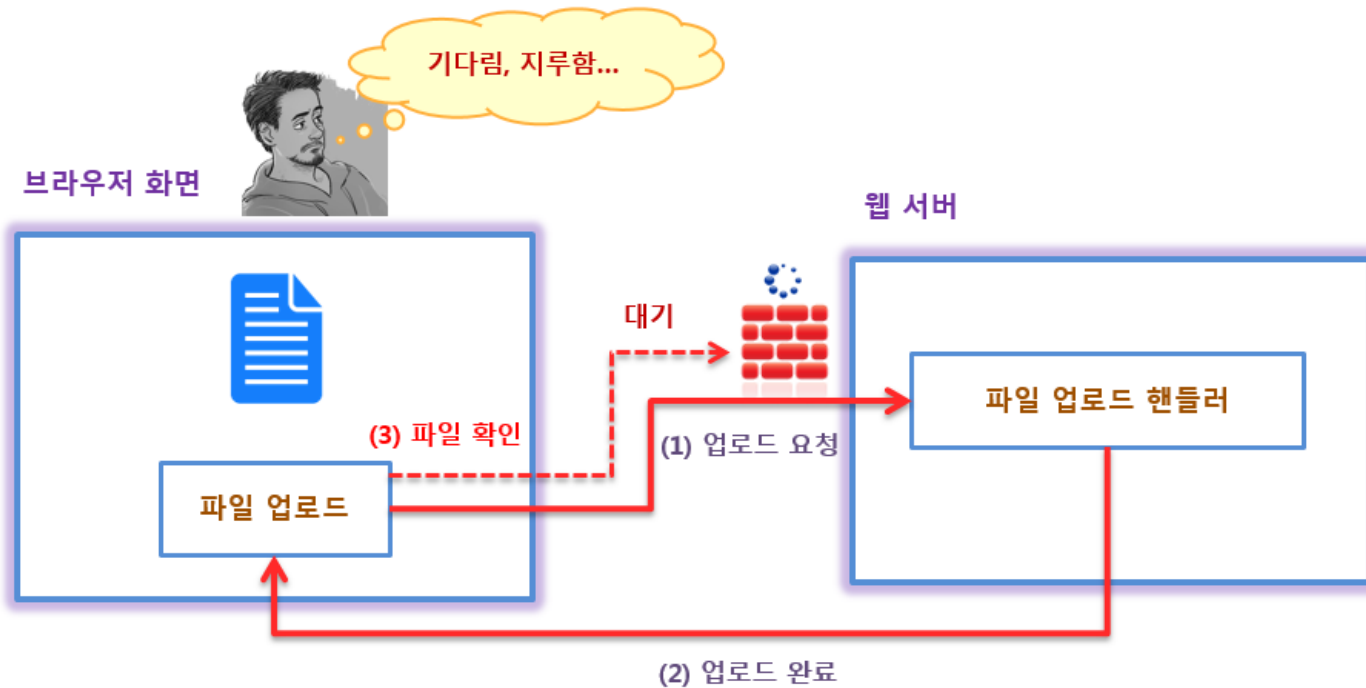
● 클라이언트

- 웹 사용자
- 웹 브라우저를 통해서 URL을 입력하여 특정 웹 서버의 문서 요청 및 확인



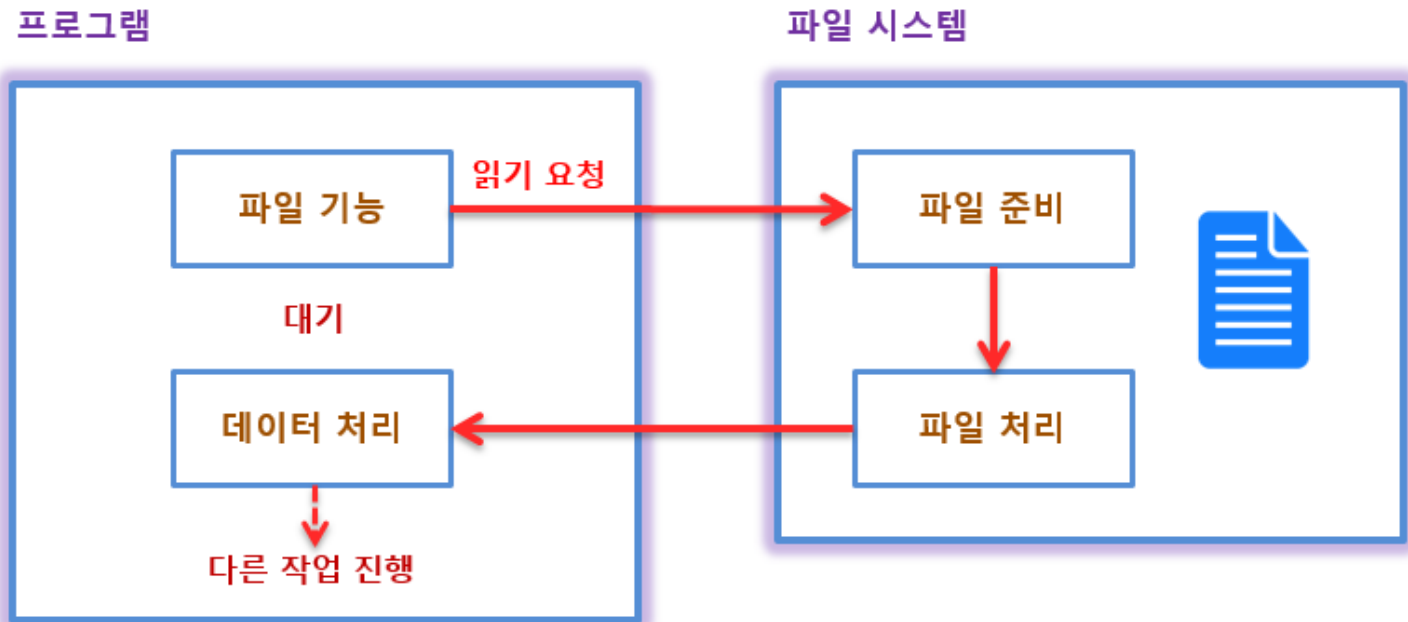
노드란 무엇일까?

- 노드제이에스(Node.js)는 노드라고도 부름
- 파일 업로드 시의 문제를 해결하기 위해 새로운 방식의 서버 도구를 만들기 시작함



동기 입출력 방식

- 파일 내용을 다 읽고 난 후에 다른 일을 처리함

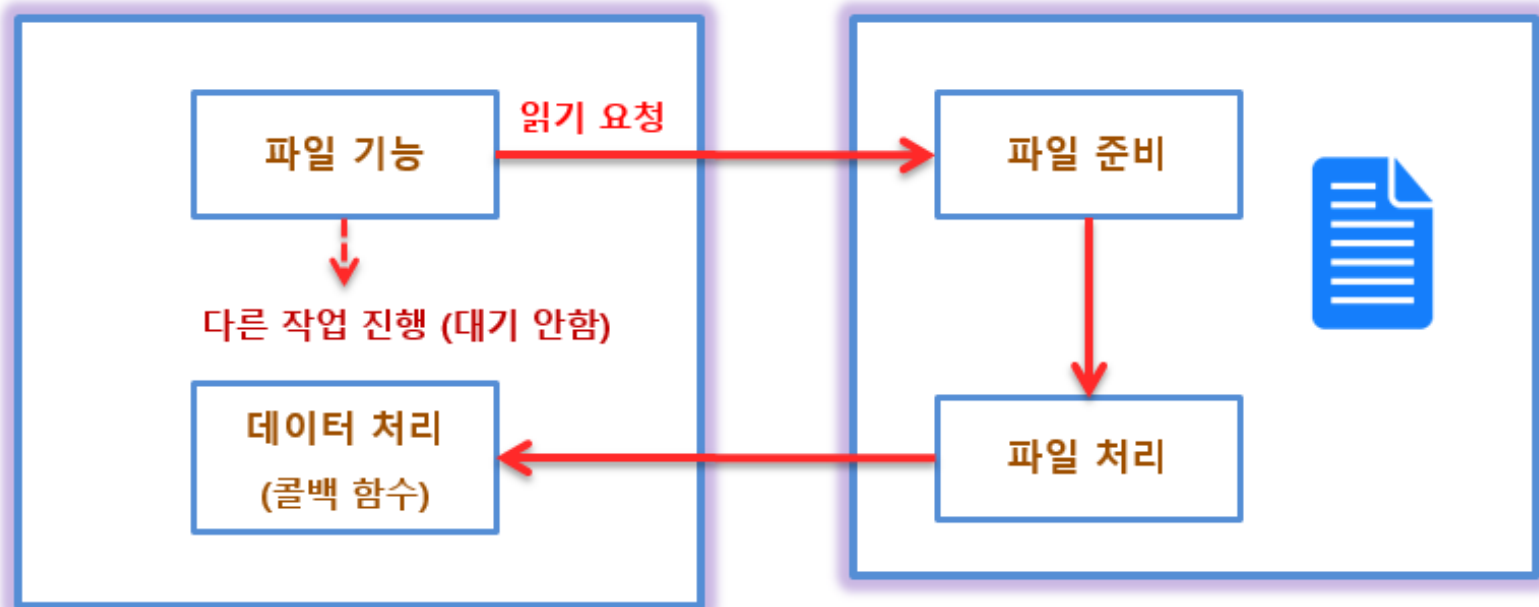


노드의 비동기 입출력 방식

- Non-Blocking IO
 - 하나의 요청 처리가 끝날 때까지 기다리지 않고 다른 요청을 동시에 처리
- 콜백 함수가 사용됨

프로그램

파일 시스템



동기 방식 vs 비동기 방식

● 동기 방식

- 파일의 내용을 읽어 함수의 결과값으로 반환함
- 결과값이 반환될 때까지 대기함

● 비동기 방식

- 파일 내용을 읽은 후 콜백 함수로 반환함
- 파일 읽기가 시작되면 즉시 다음 실행 순서로 넘어감

블로킹 방식

```
var contents = file.read('a.txt');
```

대기

```
doShow(contents);
```

```
var result = doAdd(10, 10);
```



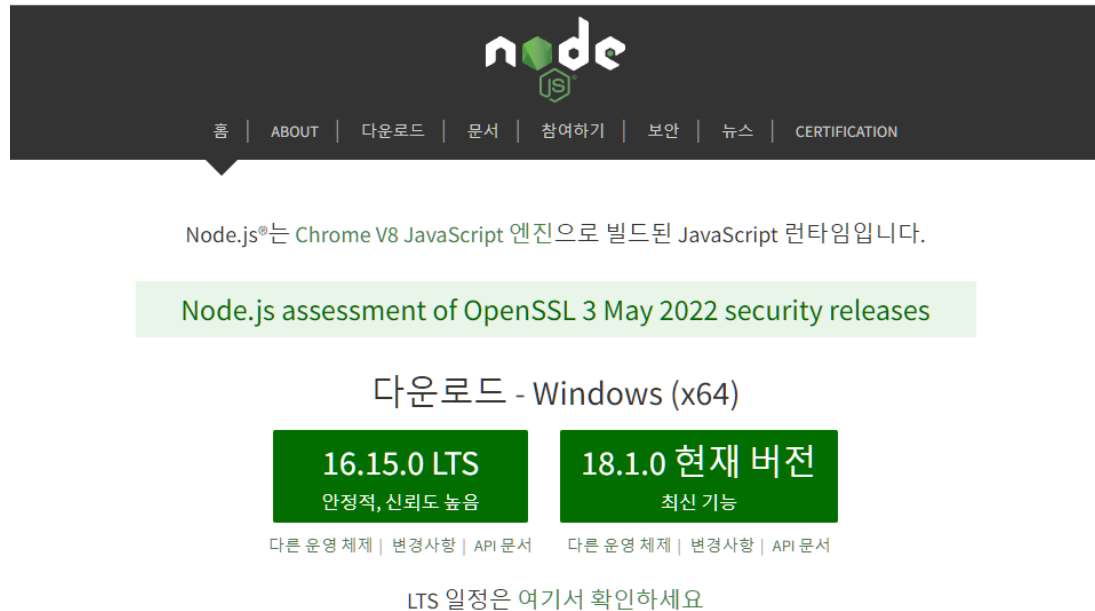
a.txt

논-블로킹 방식

```
file.read('a.txt', function(contents) {  
    doShow(contents);  
});  
  
var result = doAdd(10, 10);
```

Node.js 설치

- <https://nodejs.org/ko/>
- LTS 버전과 Current 버전 존재
 - LTS 버전은 안정성을 위해 기존 기능을 일정 기간 동안 유지
 - Current는 최신 버전



The image shows a screenshot of the Node.js website. At the top is the Node.js logo. Below it is a navigation bar with links: 홈, ABOUT, 다운로드, 문서, 참여하기, 보안, 뉴스, and CERTIFICATION. A text block states: "Node.js®는 Chrome V8 JavaScript 엔진으로 빌드된 JavaScript 런타임입니다." Below this is a green banner with the text "Node.js assessment of OpenSSL 3 May 2022 security releases". The "다운로드 - Windows (x64)" section features two green buttons: "16.15.0 LTS" with the subtext "안정적, 신뢰도 높음" and "18.1.0 현재 버전" with the subtext "최신 기능". Below these buttons are links for "다른 운영 체제 | 변경사항 | API 문서" for both versions. At the bottom, it says "LTS 일정은 여기서 확인하세요".

Node.js®는 Chrome V8 JavaScript 엔진으로 빌드된 JavaScript 런타임입니다.

Node.js assessment of OpenSSL 3 May 2022 security releases

다운로드 - Windows (x64)

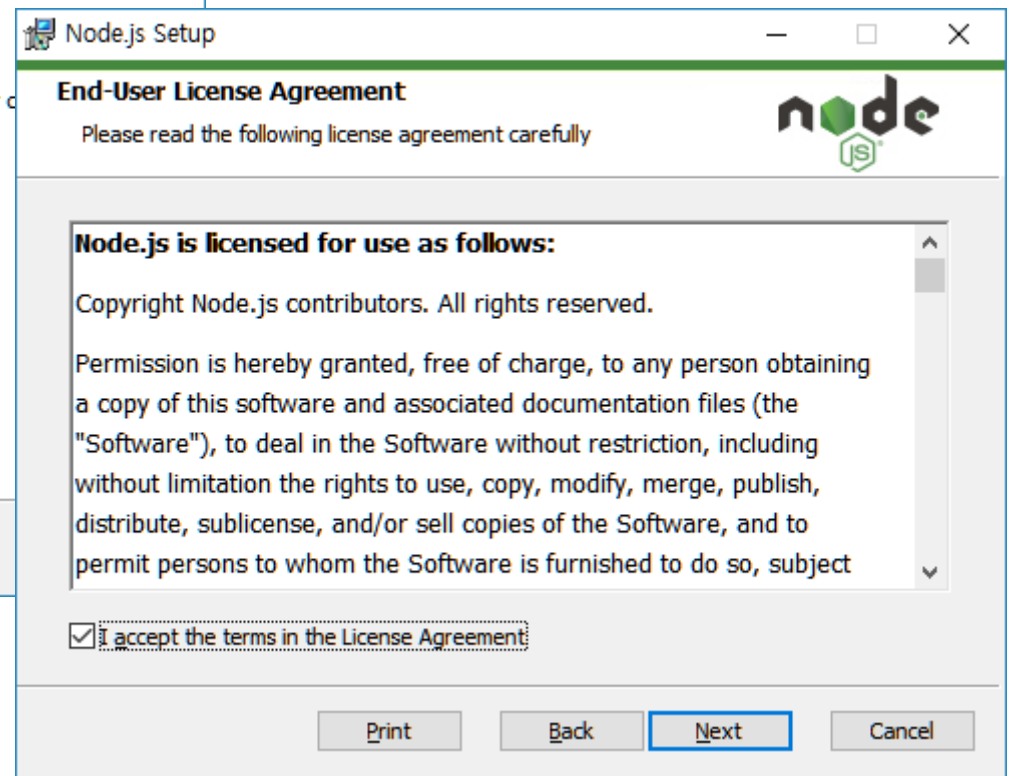
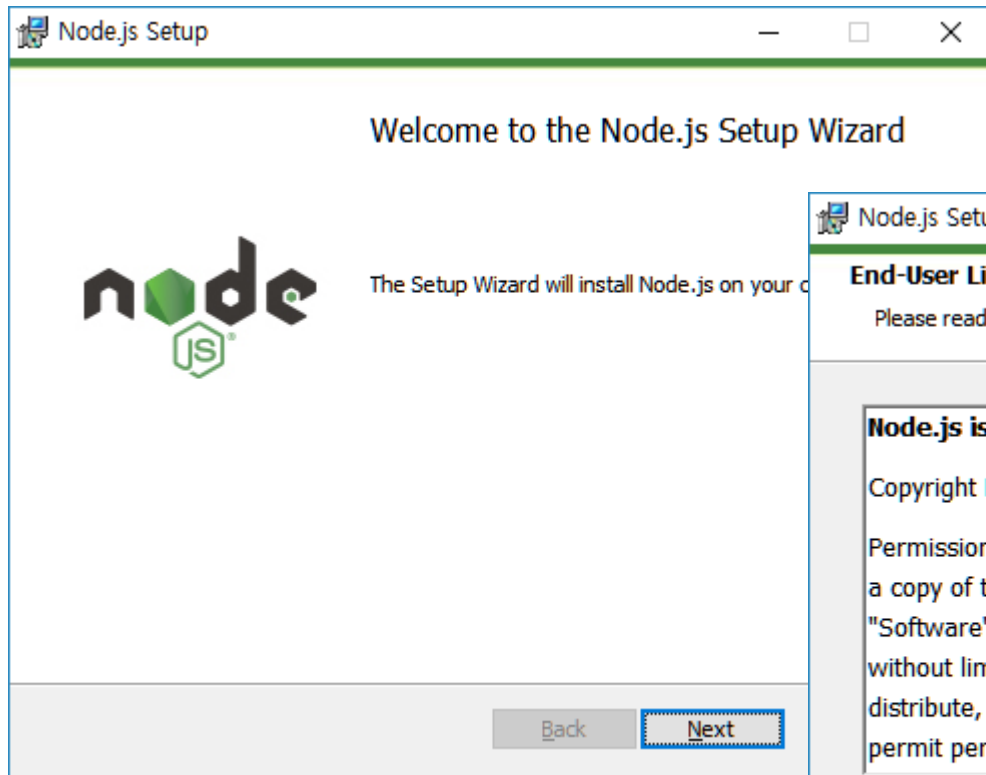
16.15.0 LTS
안정적, 신뢰도 높음

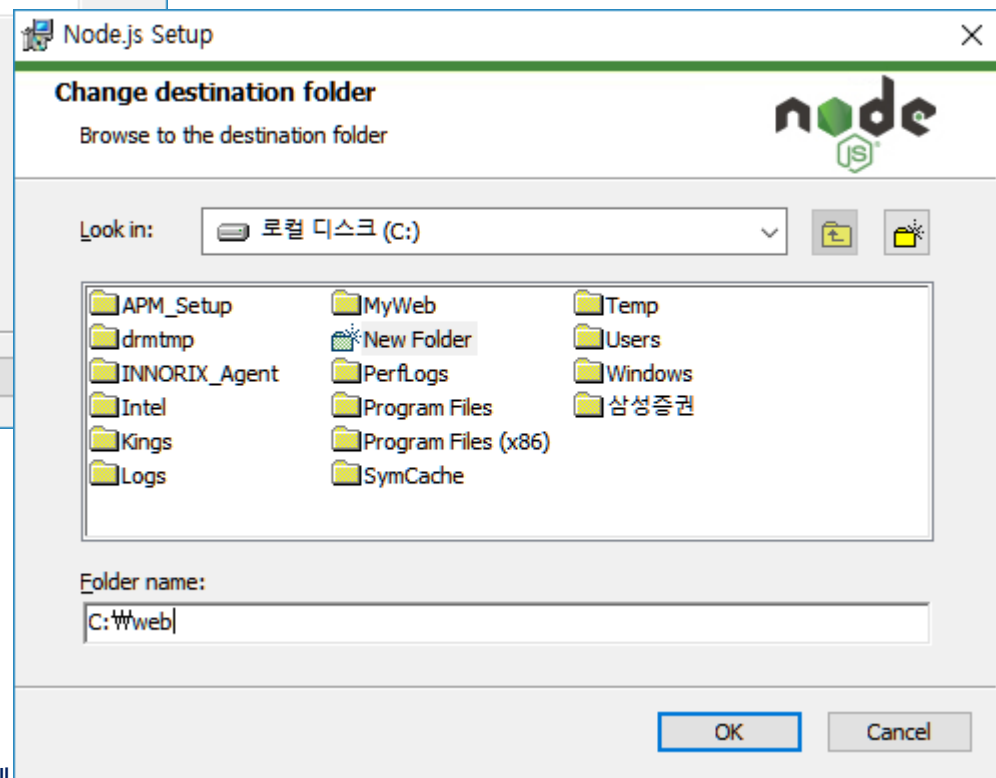
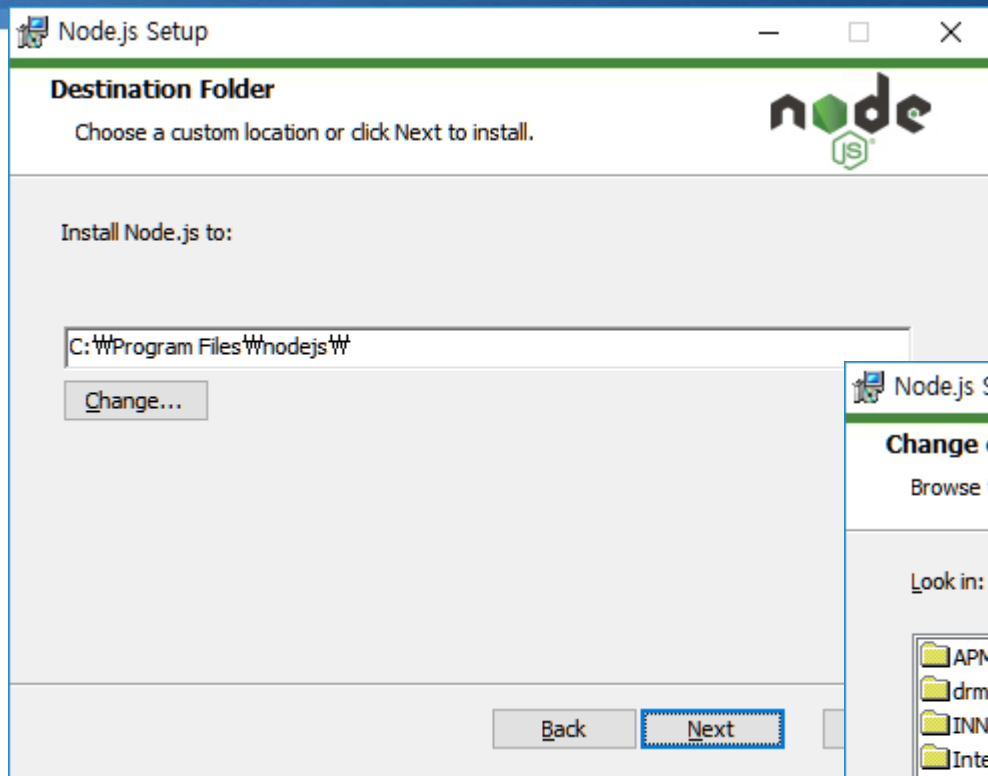
18.1.0 현재 버전
최신 기능

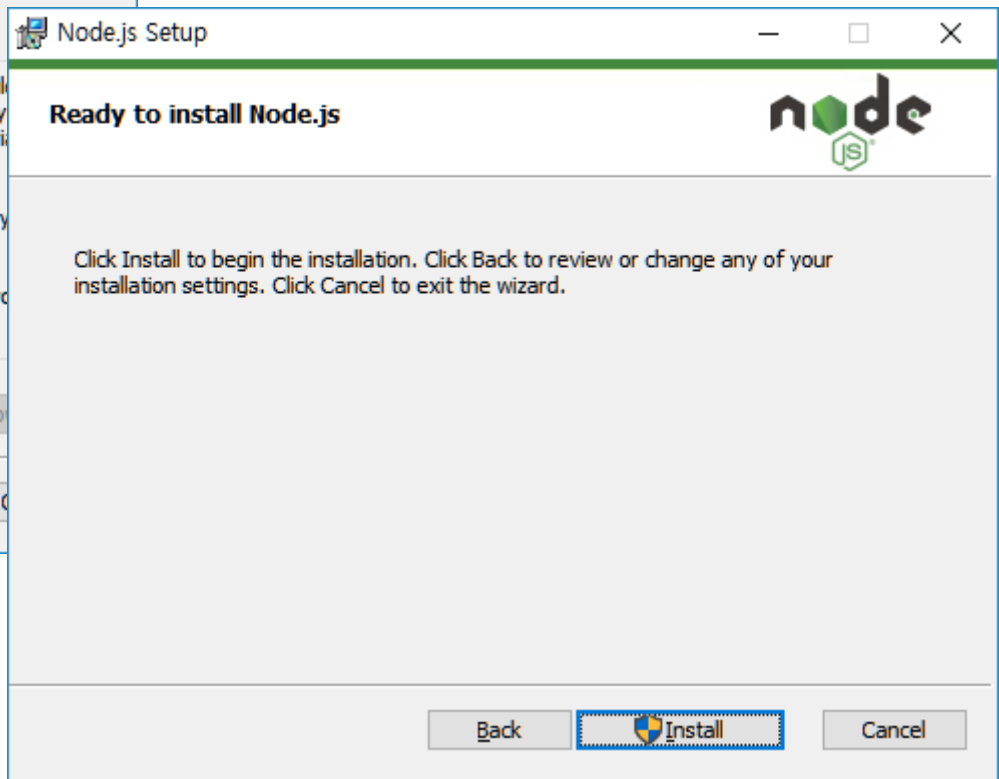
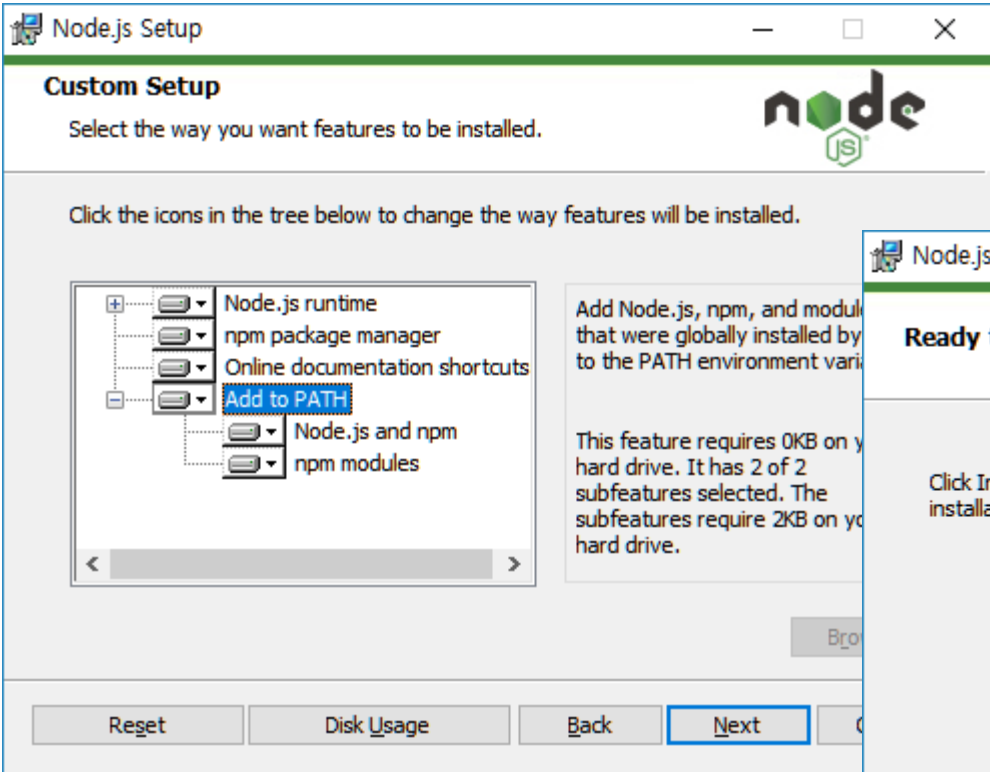
다른 운영 체제 | 변경사항 | API 문서 다른 운영 체제 | 변경사항 | API 문서

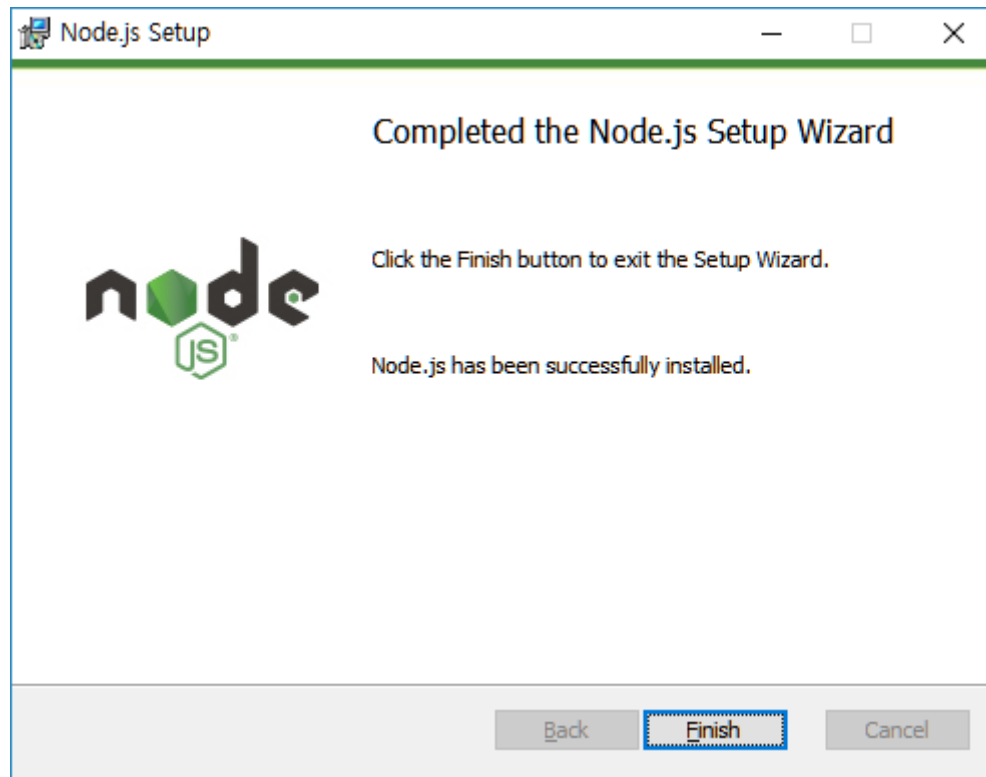
LTS 일정은 여기서 확인하세요











● 노드 설치 확인

C:\ 선택 관리자: 명령 프롬프트

```
c:\Users\WSYPARK>node -v  
v16.15.0
```

```
c:\Users\WSYPARK>npm -v  
8.5.5
```

```
c:\Users\WSYPARK>_
```



노드 모듈 설치



● 모듈

- 필요한 일부 기능을 **별도의 js 파일로 작성해** 놓은 것
 - 노드 내장 모듈 및 npm으로 설치한 외장 모듈
 - http 모듈: 웹 서버 객체 생성
 - url 모듈: url 주소 문자열가 url 객체간 상호 변환 기능
 - Fs 모듈: 파일 다루기 위한 모듈 등
 - 사용자 정의 모듈
 - ex) module1.js
- 모듈 호출: **require()** 함수 사용
 - 내장 모듈 및 npm 설치 모듈: 상대 경로 없이 사용
 - **require('http'), require('url')**
 - 사용자 정의 모듈: 상대 경로 및 폴더 지정 가능
 - **require('./module1')**

● 패키지

- 여러 개의 모듈을 합쳐 놓은 것



Express 모듈



- express 모듈

- http 모듈을 기반으로 웹 서버의 동작을 쉽게 구현할 수 있도록 작성된 모듈
- express에서 제공하는 **미들웨어**와 **라우터**를 사용하여 훨씬 간단하고 쉽게 웹 서버 구축 가능

- 미들웨어

- 익스프레스에서 웹 요청과 응답에 필요한 처리를 진행할 수 있도록 분리된 독립된 함수

- 라우터

- 클라이언트의 요청 패스를 보고 이를 처리할 수 있는 곳으로 기능을 전달해 주는 역할



기본 모듈 및 패키지 설치

- 모듈 및 패키지 설치: **npm**
 - node package manager
 - 프로그래머가 미리 개발하여 올려 둔 패키지를 찾아 설치 방법 제공
- 명령 프롬프트 (cmd) 실행
- 노드 설치 폴더 이동
 - **cd c:\web**
 - **npm init**
 - 엔터, 엔터, 엔터,....., yes
 - package.json 생성됨



express 모듈 설치



- 모듈 설치

- express 모듈

- **npm install express --save**

- static 미들웨어

- 특정 폴더의 파일들을 특정 패스로 접근할 수 있도록 허용

- **npm install serve-static --save**



app.js

// express 기본 모듈 불러오기

```
var express = require('express'), http = require('http');
```

// express 미들웨어 불러오기

```
var static = require('serve-static');
```

// express 객체 생성

```
var app = express();
```

// 기본 속성 설정

```
app.set('port', process.env.PORT || 8080);
```

```
app.set('host', '127.0.0.1');
```

//static 서버 미들웨어 사용

```
app.use(static(__dirname)); // 현재 폴더에 대한 정적 접근 허용
```

// express 서버 시작

```
http.createServer(app).listen(app.get('port'), app.get('host'), () => {  
    console.log('Express server running at ' + app.get('port') + app.get('host'));  
});
```



웹 서버 실행



- 웹 폴더(c:\web)로 파일 복사
 - jquery.html, jquery.css, jquery.js 및 기타 이미지 파일
- 웹 서버 실행: 명령 프롬프트 (cmd)
 - 웹 폴더로 이동
 - `cd c:\web`
 - `node app.js`
- 크롬 브라우저
 - <http://localhost:8080/jquery.html>
 - <http://본인의 ip주소:8080/jquery.html>



app.js (확장)



```
// express 기본 모듈 불러오기
var express = require('express'), http = require('http'), path = require('path');

// express 미들웨어 불러오기
var static = require('serve-static');

// express 객체 생성
var app = express();
var router = express.Router();

// 기본 속성 설정
app.set('port', process.env.PORT || 8080);
app.set('host', '127.0.0.1');

// 웹 폴더에 저장된 모든 웹 페이지에 대한 정적 참조 허용
app.use(static(__dirname));
```



app.js



```
app.use(express.urlencoded());  
app.use(express.json());  
  
// http 서버 생성  
http.createServer(app).listen(app.get('port'), app.get('host'), () => {  
    console.log('Express server running at ' + app.get('port') + app.get('host'));  
});
```



웹 프로그래밍

14. 웹 서버 구축 (2)

Node.js로 웹 서버 만들기

● 익스프레스 서버 만들기

```
var express = require('express');  
// 익스프레스 객체 생성  
var app = express();  
// 기본 포트를 app 객체에 속성으로 설정  
app.set('port', process.env.PORT || 8080);  
// Express 서버 시작  
http.createServer(app).listen(app.get('port'), function(){  
  console.log('익스프레스 서버를 시작했습니다 : ' + app.get('port'));  
});
```



Express 객체



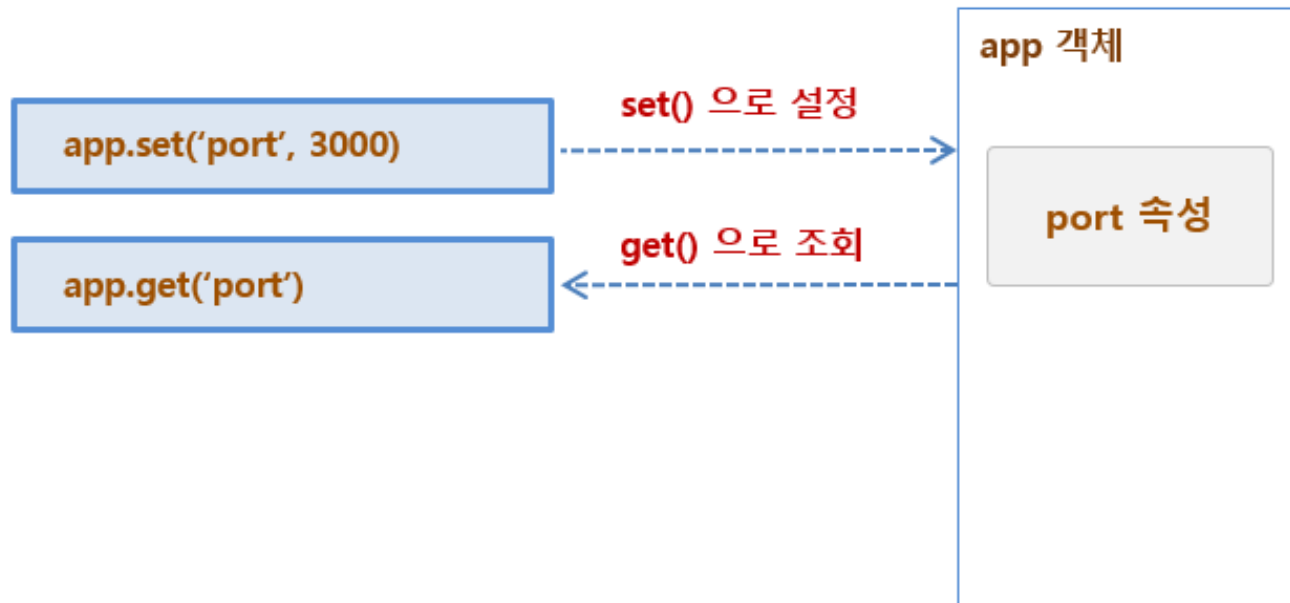
● 주요 메소드

메소드 이름	설명
set(name, value)	서버 설정을 위한 속성을 지정한다. set() 메소드로 지정한 속성은 get() 메소드로 확인한다.
get(name)	서버 설정을 위해 지정한 속성을 읽어들인다.
use([path,] function [, function...])	미들웨어 함수를 사용한다.
get([path,] function)	특정 패스로 요청된 정보를 처리한다.



포트 설정

```
// 기본 포트를 app 객체에 속성으로 설정  
app.set('port', process.env.PORT || 3000);
```



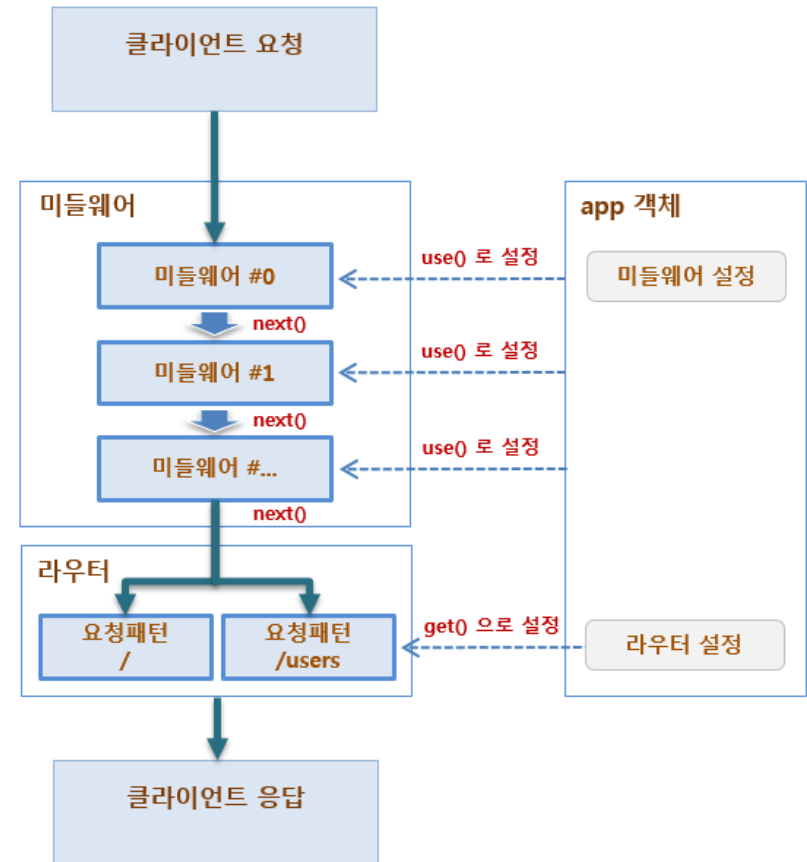
미들웨어 사용하기

● 미들웨어

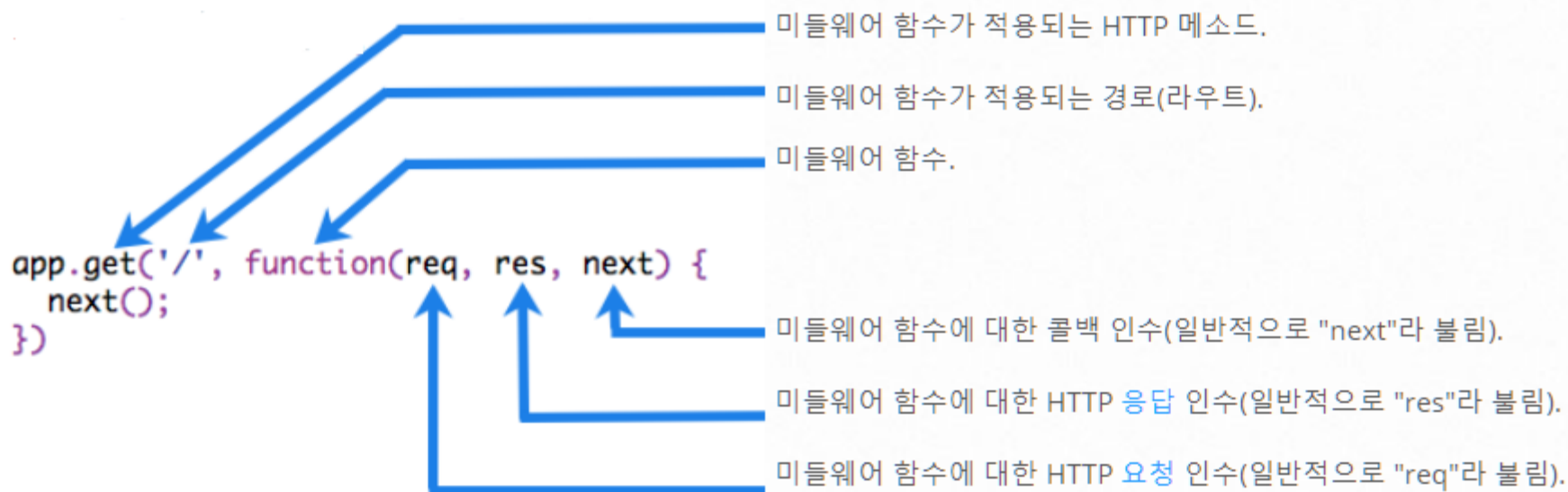
- 서버가 수행해야 할 동작 함수
- 요청 오브젝트(req), 응답 오브젝트(res) 및 다음의 미들웨어 함수에 대한 액세스 권한을 갖는 함수
- use() 메소드로 등록
- use() 메소드로 등록된 순서대로 수행

● 라우터

- 서버에 요청되는 **요청 패스(url 주소)**에 따라 분기하여 처리
- get() 메소드로 등록



사용자 요청 처리



사용자 요청 처리



● 루트 패스에 대한 요청 처리

```
app.get('/', function (req, res) {  
  res.send('Hello World!');  
});
```

● 미들웨어를 통한 요청 처리

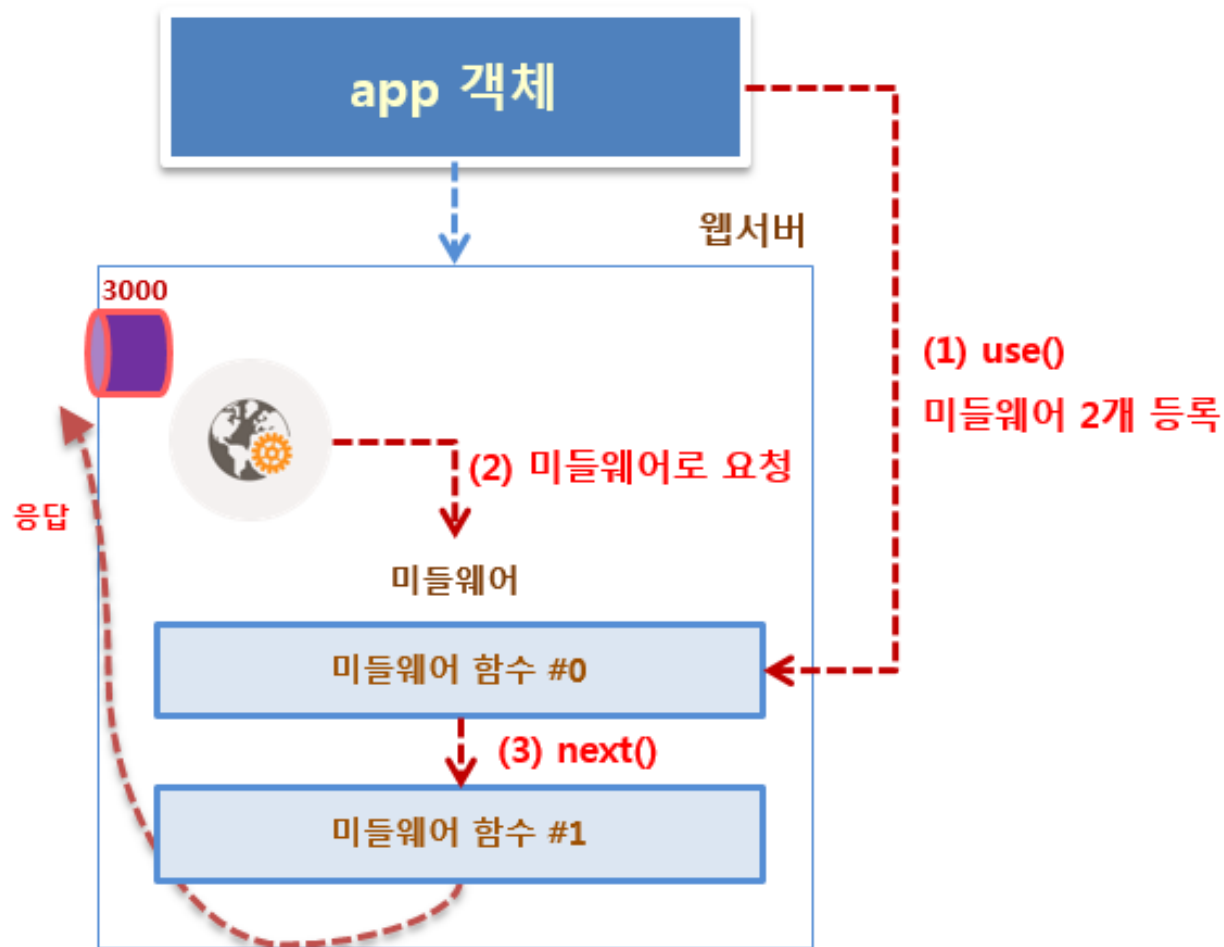
```
var myLogger = function (req, res, next) {  
  console.log('LOGGED');  
  next();  
};
```

```
app.use(myLogger);
```

```
app.get('/', function (req, res) {  
  res.send('Hello World!');  
});
```



여러 개의 미들웨어 등록



미들웨어 등록



- 여러 개의 미들웨어 등록
 - 등록 순서대로 수행
 - 모든 미들웨어를 수행하기 위해서는 next() 메소드 사용!

```
app.use(function(req, res, next) {  
  console.log('첫 번째 미들웨어에서 요청을 처리함.');  
  req.user = 'mike';  
  next();  
});
```

```
app.use('/', function(req, res, next) {  
  console.log('두 번째 미들웨어에서 요청을 처리함.');  
  res.writeHead('200', {'Content-Type': 'text/html; charset=utf8'});  
  res.end('<h1>Express 서버에서 ' + req.user + '가 응답한 결과입니다.</h1>');});
```



익스프레스의 요청 객체 및 응답 객체

메소드 이름	설명
send([body])	클라이언트에 응답 데이터를 전송한다. HTML 문자열, Buffer 객체, JSON 객체, JSON 배열 데이터 전송 가능하다.
status(code)	HTTP 상태 코드를 반환한다. 상태 코드는 end()나 send() 같은 전송 메소드를 추가로 호출해야 전송 가능하다.
sendStatus(statusCode)	HTTP 상태 코드를 반환한다. 상태 코드는 상태 메시지와 함께 전송된다.
redirect([status,] path)	웹 페이지 경로를 강제로 이동시킨다.
render(view [, locals][, callback]	뷰 엔진을 사용해 문서를 만든 후 전송한다.

```
app.use(function(req, res, next) {  
  console.log('첫 번째 미들웨어에서 요청을 처리함.');
```

```
  res.send({name:'소녀시대', age:20});  
});
```



redirect() 메소드



● 다른 페이지로 이동

```
app.use(function(req, res, next) {  
  console.log('첫 번째 미들웨어에서 요청을 처리함.');  
  res.redirect('http://google.co.kr');  
});
```



Router를 사용한 요청 라우팅

- 사용자 **요청 패스(url 주소)**에 따른 미들웨어 등록 기능

```
var router = express.Router();  
  
router.route('/process/login').post(function(req, res) {  
... });  
  
app.use('/', router);
```



Router 객체 주요 메소드



메소드 이름	설명
get(callback)	GET 방식으로 특정 패스 요청이 발생했을 때 사용할 콜백 함수 지정
post(callback)	POST 방식으로 특정 패스 요청이 발생했을 때 사용할 콜백 함수 지정
put(callback)	PUT 방식으로 특정 패스 요청이 발생했을 때 사용할 콜백 함수 지정
delete(callback)	DELETE 방식으로 특정 패스 요청이 발생했을 때 사용할 콜백 함수 지정
all(callback)	모든 요청 방식에 대해서, 특정 패스 요청이 발생했을 때 사용할 콜백 함수를 지정



오류 페이지



- 등록되지 않은 요청 패스일 경우 오류 페이지 표시

```
app.all('*', function(req, res) {  
  res.status(404).send('<h1>ERROR - 페이지를 찾을 수 없습니다.</h1>');  
});
```

